Coffee Shop Recommender System

Full Technical Report & Documentation

Version 1.0.0

A hybrid implicit-feedback recommender system that leverages the Google Maps API, keyword-based sentiment analysis, collaborative filtering, and proximity scoring to recommend the top 8 coffee shops based on a user's location.

1. Project Overview

The Coffee Shop Recommender System is a full-stack web application that helps users discover the best coffee shops near any location in the world. Users can search by city name (e.g., Kegalle, New York, Berlin) or use their device's GPS, and the system recommends the top 8 coffee shops using a hybrid scoring algorithm that combines:

- Content-based scoring — Google ratings, review volume, review sentiment, and price level
- Collaborative filtering — User-based implicit-feedback analysis via cosine similarity
- Location proximity — Exponential distance-decay scoring
- New-shop boosting — Surfacing recently added shops for discovery

Each recommended shop includes real-time data fetched from the Google Maps Places API: photos, reviews, ratings, opening hours, contact info, and Google Maps directions.

Key Features

Feature	Description
City Search	Search any city/address worldwide with autocomplete suggestions
GPS Location	One-click geolocation detection via browser API
AI Scoring	Hybrid 4-pillar recommender algorithm (content + collaborative + proximity + newness)
Photo Gallery	Scrollable photo carousel sourced from Google Maps
Reviews & Ratings	Full Google reviews with author info, timestamps, and star ratings
Interactive Map	Dark-themed Leaflet map with custom markers on all pages
Directions	One-click Google Maps navigation link
Authentication	JWT-based auth with token expiry, protected routes, and auto-logout
Premium UI	Dark theme, glassmorphism, micro-animations, responsive design

2. Quick Start Guide

2.1 Prerequisites

Software	Version	Purpose
Python	3.9+	Backend runtime
Node.js	18+	Frontend runtime
npm	9+	Frontend package manager
Git	Any	Version control

You will also need a Google Maps API Key with the following APIs enabled: Places API, Directions API.

2.2 Backend Setup

```
# 1. Navigate to the backend directory
cd "Coffee Rec System/BackEnd"

# 2. Create and activate Python virtual environment
python -m venv venv
source venv/bin/activate   # macOS/Linux
venv\Scripts\activate       # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Create .env file in BackEnd/ directory:
GOOGLE_MAPS_API_KEY=your_google_maps_api_key_here
SECRET_KEY=your_jwt_secret_key_here
DATABASE_URL=sqlite:///./coffee_rec.db

# 5. Start the backend server
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Backend API: http://localhost:8000 | Swagger Docs: http://localhost:8000/docs

2.3 Frontend Setup

```
cd "Coffee Rec System/FrontEnd"
npm install
npm run dev
# Frontend available at: http://localhost:5173
```

2.4 Dependencies

Backend (requirements.txt)

Package	Purpose
fastapi	Web framework
uvicorn	ASGI server
sqlalchemy	ORM (Object-Relational Mapper)
python-dotenv	Environment variable management

Package	Purpose
googlemaps	Google Maps API Python client
passlib[bcrypt]	Password hashing
python-jose[cryptography]	JWT token encoding/decoding
scikit-learn	Machine learning utilities
numpy	Numerical computing
scipy	Scientific computing

Frontend (package.json)

Package	Purpose
react / react-dom	UI library and DOM renderer
react-router-dom	Client-side routing
axios	HTTP client for API calls
framer-motion	Animation library
react-icons	Icon library (Feather icons)
leaflet / react-leaflet	Map rendering library

3. Technology Stack

3.1 Backend Technologies

Technology	Version	Purpose
Python	3.9+	Primary backend language
FastAPI	Latest	Async web framework with auto-generated Swagger docs
Uvicorn	Latest	ASGI server for running FastAPI
SQLAlchemy	Latest	ORM for database models and queries
SQLite	Built-in	Lightweight relational database (default)
Pydantic	v2	Data validation and serialization
python-jose	Latest	JWT creation and validation
Passlib + bcrypt	Latest	Password hashing (SHA-256 pre-hash -> bcrypt)
googlemaps	Latest	Google Maps API Python client
NumPy	Latest	Vector operations for collaborative filtering

3.2 Frontend Technologies

Technology	Version	Purpose
React	18+	Component-based UI framework
Vite	8.x	Fast build tool and dev server with HMR
React Router v6	Latest	Client-side routing with protected routes
Axios	Latest	HTTP client with interceptor support
Framer Motion	Latest	Declarative animation library
Leaflet	Latest	Interactive map rendering
React Icons	Latest	Feather icon set
Vanilla CSS	—	Custom design system with CSS variables
Google Fonts	—	Outfit (display) + Inter (body) typography

3.3 External APIs

API	Provider	Usage
Places API — Nearby Search	Google Maps	Discover coffee shops within a radius
Places API — Place Details	Google Maps	Fetch reviews, photos, hours, contact info
Places API — Place Photos	Google Maps	Retrieve shop photographs
Nominatim Geocoding	OpenStreetMap	Convert city names to lat/lng coordinates
CartoDB Dark Tiles	CARTO	Dark-themed map tiles for Leaflet

4. System Architecture

The application follows a clean client-server architecture with a React SPA on the frontend communicating with a FastAPI backend via a Vite proxy. The backend integrates with Google Maps APIs for real-time shop data and persists information in SQLite using SQLAlchemy ORM.

Data Flow Summary

- User enters a city name → Frontend calls Nominatim API → converts to lat/lng
- Frontend sends lat/lng → GET /api/shops/recommend?lat=X&lng=Y
- Backend scrapes Google Maps → Nearby Search API → caches shops in SQLite
- Recommender engine scores → Hybrid algorithm produces top 8
- Frontend displays results → Shop cards with photos, ratings, distance
- User clicks a shop → GET /api/shops/{place_id} → Full details from Google + DB
- User clicks Directions → Google Maps deep-link opens in new tab
- Interactions are logged → Implicit feedback feeds back into collaborative filtering

5. Backend Architecture

5.1 Directory Structure

```
BackEnd/
├── main.py                     # FastAPI application entry point
├── requirements.txt            # Python dependencies
├── .env                        # Environment variables (not committed)
├── coffee_rec.db               # SQLite database (auto-created)
└── app/
    ├── database.py             # SQLAlchemy engine, session, Base
    ├── models.py               # ORM models
    ├── schemas.py              # Pydantic request/response schemas
    ├── routers/
    │   ├── auth.py             # /api/auth/register, /api/auth/login
    │   └── shops.py            # /api/shops/* endpoints
    └── services/
        ├── auth_service.py     # JWT + password hashing
        ├── google_maps.py      # Google Maps API integration
        └── recommender.py      # Hybrid recommender algorithm
```

5.2 Application Lifecycle

- Startup: Creates all database tables using SQLAlchemy's Base.metadata.create_all()
- Runtime: Serves API requests via Uvicorn ASGI server
- Shutdown: Logs shutdown message and releases resources

5.3 CORS Configuration

The backend allows all origins (*) during development. For production, restrict allow_origins to your frontend domain.

6. Recommender System — Deep Dive

6.1 Overview

The recommender is a hybrid implicit-feedback system that produces a final score for each coffee shop by combining four independent scoring pillars, each normalised to a 0-1 range:

```
Final Score = (alpha x Content Score) + (beta x Collaborative Score) + (gamma x Proximity Score) +
(delta x Newness Boost)
```

Symbol	Weight	Pillar	Description
alpha (ALPHA)	0.35	Content-Based	Shop quality signals
beta (BETA)	0.25	Collaborative Filtering	Similar-user preferences
gamma (GAMMA)	0.30	Location Proximity	Distance decay
delta (DELTA)	0.10	New-Shop Boost	Discovery incentive

6.2 Pillar 1: Content-Based Scoring (alpha = 0.35)

This pillar evaluates each shop's intrinsic quality using data from Google Maps. The content score is a weighted combination of four sub-factors:

```
Content Score = (0.40 x Rating) + (0.25 x Popularity) + (0.20 x Sentiment) + (0.15 x Price)
```

Rating Component (40%)

```
rating_score = min(shop.rating / 5.0, 1.0)
```

The Google rating (0-5 stars) is normalised to 0-1. A shop rated 4.5 stars scores 0.90.

Popularity Component (25%)

```
pop = log1p(shop.total_ratings) / log1p(max_total_across_all_shops)
```

Uses logarithmic scaling (log1p) to prevent shops with thousands of reviews from completely dominating. This creates a fairer distribution.

Review Sentiment Component (20%)

A keyword-based sentiment analyser that scans all review texts for positive and negative words.

- Positive keywords (26): great, excellent, amazing, wonderful, fantastic, love, best, perfect, delicious, friendly, cozy, awesome, recommend, favourite, beautiful, lovely, superb, outstanding, good, nice, clean, comfortable, warm, welcoming, pleasant...
- Negative keywords (16): bad, terrible, awful, worst, dirty, rude, slow, horrible, disgusting, disappointing, overpriced, cold, stale, poor, avoid, never

```
sentiment = positive_count / (positive_count + negative_count)
```

If no sentiment words are found, a neutral default of 0.5 is used. This approach is lightweight, requires no ML model, and runs in O(n).

Price Level Component (15%)

```
price_score = max(0, 1.0 - abs(price_level - 2) * 0.25)
```

Price Level	Score	Interpretation
0 (free/unknown)	0.50	Penalised — likely missing data
1 ($)	0.75	Budget-friendly
2 ($$)	1.00	Sweet spot
3 ($$$)	0.75	Upscale
4 ($$$$)	0.50	Premium/luxury

6.3 Pillar 2: Collaborative Filtering (beta = 0.25)

This pillar implements user-based collaborative filtering on implicit interaction data.

Interaction Weights

Interaction Type	Weight	Signal Strength
view	1.0	Mild interest
read_reviews	2.0	Moderate interest
click_direction	3.0	Strong intent to visit

How It Works

- Build User-Item Interaction Matrix: For every user, a row vector is created where each column represents a coffee shop. Cells contain weighted interaction signals.
- Compute Cosine Similarity: similarity(A,B) = dot(A,B) / (||A|| x ||B||). Users who interacted with the same shops in similar ways have high similarity.
- Score Shops: score(shop) = Sum(similarity(current_user, similar_user) x similar_user_interaction_weight)
- Normalise all collaborative scores to 0-1.

Cold-Start Handling

When a user has no interaction history (new users), the collaborative score returns empty, and the system gracefully falls back to the other three pillars (content + proximity + newness).

6.4 Pillar 3: Location Proximity (gamma = 0.30)

Uses exponential distance decay to favour closer shops:

```
proximity_score = e^(-distance_km / 3.0)
```

Distance	Score	Interpretation
0 km	1.00	Right here
1 km	0.72	Walking distance
3 km	0.37	Short drive
5 km	0.19	Moderate drive
10 km	0.04	Far — heavily penalised

Distance is calculated using the Haversine formula — the standard method for computing great-circle distance between two points on a sphere. R = 6371 km (Earth's radius).

6.5 Pillar 4: New-Shop Boost (delta = 0.10)

Shops added to the database within the last 7 days receive a linear decay bonus:

```
bonus = max(0, 1.0 - age_in_days / 7)
```

Shop Age	Bonus	Purpose
Day 0 (just added)	1.00	Maximum visibility
Day 3	0.57	Still boosted
Day 7+	0.00	Normal operation

This prevents the 'rich-get-richer' problem where popular established shops dominate all recommendations, ensuring new coffee shops get a fair chance of being discovered.

6.6 Final Score Calculation — Example

Consider a shop with: Google rating 4.5, 120 reviews, positive sentiment 0.80, price level 2, new user (no history), distance 2 km, added 2 days ago:

```
Content  = 0.40x(4.5/5) + 0.25x(log(121)/log(max)) + 0.20x0.80 + 0.15x1.0
         ~ 0.360 + 0.188 + 0.160 + 0.150 = 0.858

Collab   = 0.0  (new user, no history)

Proximity = e^(-2/3) ~ 0.513

Newness  = 1.0 - 2/7 ~ 0.714

Final    = 0.35x0.858 + 0.25x0.0 + 0.30x0.513 + 0.10x0.714
         = 0.300 + 0.000 + 0.154 + 0.071 = 0.525
```

7. API Reference

7.1 Authentication Endpoints

POST /api/auth/register

Create a new user account.

Parameter	Type	Required	Description
email	string	Yes	User email address
password	string	Yes	Password (min. 6 characters)

POST /api/auth/login

Authenticate an existing user. Returns a JWT access token.

7.2 Shop Endpoints

GET /api/shops/recommend

Get personalised coffee shop recommendations.

Parameter	Type	Required	Default	Description
lat	float	Yes	—	Latitude
lng	float	Yes	—	Longitude
radius	int	No	5000	Search radius (metres)

GET /api/shops/{place_id}

Get full details for a single shop. Response includes photo_urls[], reviews[], opening_hours, website, phone_number, business_status, types[].

GET /api/shops/{place_id}/photos

Get all photo URLs for a shop.

GET /api/shops/{place_id}/reviews

Paginated reviews with page and page_size query parameters.

GET /api/shops/{place_id}/directions

Get a Google Maps directions URL. Requires origin_lat and origin_lng parameters.

POST /api/shops/{place_id}/interact

Log an implicit user interaction (requires authentication). interaction_type: view | read_reviews | click_direction

7.3 Health Endpoints

Endpoint	Response
GET /	Service info + version
GET /health	{"status": "ok"}

8. Frontend Architecture

8.1 Directory Structure

```
FrontEnd/src/
├── main.jsx                    # React entry point
├── App.jsx                     # Root component with Router + Providers
├── index.css                   # Global design system (CSS variables, animations)
├── services/
│   └── api.js                  # Axios instance + JWT interceptor + API functions
├── context/
│   ├── AuthContext.jsx         # JWT auth state management
│   ├── LocationContext.jsx     # Geolocation + Nominatim city search
│   └── ToastContext.jsx        # Notification system
├── components/
│   ├── Navbar.jsx              # Responsive navigation bar
│   ├── ShopCard.jsx            # Recommendation card with hover animations
│   ├── ShopMap.jsx             # Leaflet map with dark tiles + custom markers
│   ├── StarRating.jsx          # Star rating display (half-star support)
│   ├── ReviewCard.jsx          # Customer review display
│   ├── LocationPicker.jsx      # City search + GPS location input
│   ├── SkeletonCard.jsx        # Loading placeholder with shimmer
│   ├── Toast.jsx               # Slide-in notification system
│   ├── ProtectedRoute.jsx      # Auth guard (redirects to /login)
│   └── GuestRoute.jsx          # Guest guard (redirects to /home)
└── pages/
    ├── Landing.jsx             # Hero page with features showcase
    ├── Login.jsx               # Sign-in with redirect-back-to-origin
    ├── Register.jsx            # Registration with validation
    ├── Home.jsx                # Recommendation grid + map toggle
    └── ShopDetail.jsx          # Full shop page with gallery + map + reviews
```

8.2 Design System

- Dark theme: Deep navy/charcoal backgrounds (#0a0a12)
- Accent colour: Amber/gold (#d4a056) with gradient variants
- Glassmorphism: Semi-transparent cards with backdrop-filter: blur()
- Typography: Outfit (display) + Inter (body) from Google Fonts
- Animations: @keyframes for gradient shift, orb float, shimmer, spin
- CSS Variables: 30+ tokens for colours, spacing, radii, shadows, transitions

8.3 State Management

Context	Purpose	Key State
AuthContext	JWT auth lifecycle	token, user, isAuthenticated, loading
LocationContext	User location management	location {lat, lng, name}, geocoding
ToastContext	Notification queue	toasts[], auto-dismiss timers

8.4 Route Protection Strategy

Route	Component	Access
/	Landing	Public
/login	Login	GuestRoute — redirects to /home if logged in
/register	Register	GuestRoute — redirects to /home if logged in

Route	Component	Access
/home	Home	ProtectedRoute — redirects to /login if not logged in
/shop/:placeId	ShopDetail	ProtectedRoute — redirects to /login if not logged in

8.5 City Search (Geocoding Pipeline)

- User types a city name (e.g., "Kegalle")
- After 400ms debounce, LocationContext.searchCity() fires
- Calls Nominatim API: https://nominatim.openstreetmap.org/search
- Returns up to 5 autocomplete suggestions with lat/lng
- User selects a result → sets location state
- Home.jsx detects location change → calls GET /api/shops/recommend

8.6 Map Integration

Page	Map Behaviour
Home	Shows all 8 recommended shops + user location with auto-fit bounds
ShopDetail	Shows single shop + user location at zoom level 16

- Dark tiles: CartoDB dark_all basemap (matches app theme)
- Custom markers: coffee pin (amber teardrop shape) for shops
- User marker: pulsing blue ring animation
- Popups: Shop name, star rating, address, 'View Details' link

9. Authentication Pipeline

9.1 Registration Flow

```
User fills form -> POST /api/auth/register
    -> Backend hashes password (SHA-256 pre-hash -> bcrypt)
    -> Creates User record in DB
    -> Frontend auto-calls POST /api/auth/login
    -> Backend creates JWT (24h expiry, HS256)
    -> Token stored in localStorage
    -> Redirect to /home
```

9.2 Login Flow

```
User submits credentials -> POST /api/auth/login
    -> Backend verifies (SHA-256 pre-hash -> bcrypt compare)
    -> Creates JWT with user_id in payload
    -> Frontend stores token in localStorage
    -> Redirect to original page (or /home)
```

9.3 Password Security

The system uses a double-hash strategy to bypass bcrypt's 72-byte password limit:

```
User password -> SHA-256 (produces 64-char hex) -> bcrypt (produces final hash)
```

This ensures passwords of any length are securely supported.

9.4 JWT Token Lifecycle

Event	Action
Login/Register	Token created (24h TTL), stored in localStorage
Every API request	Axios interceptor attaches Authorization: Bearer <token>
Token expiry	AuthContext auto-logout timer fires, clears storage
401 API response	Axios response interceptor clears token, redirects to /login
App reload	AuthContext checks token expiry on init, clears if stale

10. Data Pipeline

10.1 Complete Request Pipeline

Step 1: Google Maps Nearby Search

- Calls places_nearby(lat, lng, radius, type='cafe', keyword='coffee')
- Fetches up to 3 pages of results (60 shops max)
- Upserts each shop into SQLite (insert or update)

Step 2: Hybrid Recommender Scoring

- Content score: rating + popularity + sentiment + price
- Collaborative score: user-item matrix -> cosine similarity
- Proximity score: haversine distance -> exponential decay
- Newness boost: linear decay over 7 days
- Final = 0.35xcontent + 0.25xcollab + 0.30xproximity + 0.10xnewness

Step 3: Build Response

- Sort by final score descending
- Take top 8
- Attach photo URLs from Google Maps Photos API
- Return JSON response

Frontend Rendering

- ShopCard grid (4 columns, responsive) with Framer Motion stagger
- Interactive Leaflet map with custom coffee-pin markers
- Grid/Map view toggle
- Click card -> /shop/:placeId -> GET /api/shops/:placeId -> Photo gallery, reviews, hours, directions, mini-map

10.2 Implicit Feedback Loop

Every user action is silently logged as an interaction, which feeds back into the collaborative filtering model:

User Action	Interaction Type	Weight
Views shop detail	view	1.0
Reads reviews	read_reviews	2.0
Clicks directions	click_direction	3.0

This creates a self-improving feedback loop — the more users interact with the system, the better its collaborative filtering becomes.

11. Database Schema

11.1 Table Descriptions

Table	Description
users	Stores email, hashed password, creation timestamp
coffee_shops	Full shop data: location, rating, photos, hours, place_id (UNIQUE)
reviews	Google review text, author, rating, timestamp (FK: shop_id)
user_interactions	Implicit feedback: view, read_reviews, click_direction (FK: user_id, shop_id)

11.2 coffee_shops Schema

Column	Type	Description
id	Integer PK	Auto-increment primary key
place_id	String UNIQUE	Google Maps place identifier
name	String	Coffee shop name
address	String	Full address
lat, lng	Float	Coordinates
rating	Float	Google rating (0-5)
total_ratings	Integer	Review count
price_level	Integer	0-4 price tier
photo_refs	JSON	List of photo references
types	JSON	Place type tags
opening_hours	JSON	Weekly hours schedule
website	String	Business website URL
phone_number	String	Contact number
business_status	String	OPERATIONAL / CLOSED_PERMANENTLY / etc.
created_at, updated_at	DateTime	Record timestamps

12. Deployment Notes

12.1 Environment Variables

Variable	Required	Default	Description
GOOGLE_MAPS_API_KEY	Yes	—	Google Maps API key with Places API enabled
SECRET_KEY	Yes	fallback	JWT signing secret (use a strong random string)
DATABASE_URL	No	sqlite:///./coffee_rec.db	SQLAlchemy database URL

12.2 Production Checklist

- Replace SQLite with PostgreSQL/MySQL for concurrent access
- Set a strong, unique SECRET_KEY
- Restrict CORS allow_origins to your frontend domain
- Enable HTTPS on both frontend and backend
- Set DEBUG=False and configure proper logging
- Add rate limiting to prevent Google Maps API abuse
- Consider Redis caching for hot Google Maps responses
- Set up a reverse proxy (Nginx) for both services

12.3 Production Commands

```
# Backend (production)
pip install gunicorn
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000

# Frontend (build for production)
cd FrontEnd
npm run build
# Serve the dist/ folder with Nginx or any static file server
```

Coffee Shop Recommender System v1.0.0 | Built with love and coffee